

Trabajo Integrador Sistemas Embebidos

UTN F.R.A

Lesme Franco, Valentin Franco, Erbino Sebastian
Escuela de Educación Técnica Secundaria N°7 (I.M.P.A)
francoagustinlesmegarcia@impatrq.com
Valentinfranco@impatrq.com
Sebastianjoseerbino@impatrq.com

Abstract—En este documento, daremos una explicación extensiva de la aplicación y alcance de sistemas embebidos, usando como herramientas FreeRTOS y lenguaje C

I. INTRODUCCIÓN

Dados los conocimientos adquiridos en el curso de sistemas embebidos, dictado en la U.T.N por el profesor Fabrizio Carlassara, hemos llevado a cabo exitosamente un trabajo integrador, aplicando todo lo aprendido sobre FreeRtos y manejo del microcontrolador LPC 845 Breakout.

II. DESARROLLO DE CONTENIDOS

Para poder llevar a cabo todas las consignas propuestas para la aprobación, empezamos a trabajar con el archivo llamado “labels.h”, estableciendo en este, de forma definida, etiquetas que lleven al fácil acceso de pines y mejoren la comprensión y entendimiento del código.

En los wrappers, divididos en wrappers.c y wrappers.h, se pretende inicializar y enmascarar las APIs del SDK de las funciones utilizadas con el fin de tener un acceso directo.

Los archivos *tareas.c* y *tareas.h* definen la estructura del sistema multitarea. En ellos se asignan prioridades, tamaños de pila y se crean los prototipos de las funciones asociadas a cada tarea. Además, se especifican sus propósitos funcionales dentro del sistema. Se incluyen también la inicialización de semáforos y colas necesarias para la sincronización y comunicación entre tareas. Esta modularización permite una organización clara del código y facilita la escalabilidad del sistema en tiempo real (FreeRTOS).

El archivo main.c realiza la configuración inicial del sistema y define el punto de entrada del programa. Se establece el reloj a 30 MHz y se inicializa la consola de depuración. A continuación, se crean múltiples tareas mediante `xTaskCreate()`, cada una con su función, nombre, tamaño de pila, prioridad y puntero de control. Estas tareas gestionan distintos módulos del sistema como sensores, display, consola, control y salidas. Finalmente, `vTaskStartScheduler()` inicia el planificador de FreeRTOS, dando comienzo a la ejecución concurrente de las tareas.

III. TAREAS

A continuación se mencionan y detallan todas las tareas realizadas con su respectivo propósito

A. Listado de tareas

- Tareas display:**
 - `tsk_display_write` – Display (escritura en pantalla)
 - `tsk_display_change` – Boton (manejo de entradas de botón)
 - `tsk_control` – Counter (control del sistema)
- Tareas de inicialización:**
 - `tsk_init` – Init
- Tareas del ADC:**
 - `tsk_adc` – ADC
- Tareas de Luminosidad:**
 - `tsk_BH1750` – BH1750 (sensor de luz)
- Tareas de Setpoint:**
 - `tsk_setpoint` – Setpoint (valor objetivo de control)
- Tareas de LED:**
 - `tsk_led_azul` – LED Azul (control del LED azul)
 - `tsk_leds_control` – LEDs (control de LEDs múltiples)
- Tareas de Buzzer:**
 - `tsk_buzzer` – Buzzer (control del zumbador)
- Tareas de Consola:**
 - `tsk_console_monitor` – Console (monitor de consola)

B. Tabla sobre hardware utilizado

Hardware utilizado en LPC 845 BREAKOUT		
Cantidad	Componente	Denominación
1	Buzzer	-
3	Botones	S1, S2 y USER
2	Potenciómetro	RV21
1	Led adicional	D1
1	Display 7 segmentos	-
1	BH1750	-
1	Sensor infrarrojo	IR
1	Led tricolor	-

C. Conceptos de FreeRTOS

¿Qué son las colas y para qué las usó en este proyecto?

En FreeRTOS, las colas (Queues) son estructuras de datos que permiten la comunicación segura entre tareas y/o interrupciones dentro del sistema operativo. Funcionan como un “puente” por donde una tarea puede enviar datos a otra, asegurando que la transferencia sea atómica (segura, sin perder información ni corromper datos, incluso si varias tareas acceden al mismo recurso).

En este caso las colas son utilizadas para:

- Transferir datos de sensores:
- Enviar información de usuario o configuración:
- Sincronizar y desacoplar tareas

¿Qué son los semáforos y para qué los uso en este proyecto?

Los semáforos en FreeRTOS son mecanismos de sincronización que permiten a las tareas y a las interrupciones coordinar el acceso a recursos compartidos o señalar la ocurrencia de eventos.

En este caso los semáforos son utilizados para:

- Sincronizar eventos de usuario:
- Proteger recursos compartidos:
- Coordinar ejecución entre ISR y tareas: Cuando una interrupción (por ejemplo, de un botón) ocurre, el ISR libera (give) el semáforo, y la tarea correspondiente se desbloquea (take) y atiende el evento.

¿Por qué tengo las prioridades?

En FreeRTOS, cada tarea tiene una prioridad que determina el orden de ejecución cuando varias tareas están listas para ejecutarse. Elegir bien las prioridades es clave para asegurar la respuesta rápida a eventos críticos y el funcionamiento fluido del sistema.

En este caso las prioridad son utilizadas de la siguiente forma:

- Tareas críticas y de respuesta rápida (como las que leen botones o gestionan el display) tienen prioridad más alta, para asegurar que el sistema responda “en tiempo real” a la interacción del usuario.
- Tareas de adquisición de datos (como la que lee el sensor de luz o el potenciómetro) tienen prioridad media, ya que su información es importante pero no requiere respuesta instantánea.
- Tareas de actualización de salidas (como PWM de LEDs) pueden tener prioridad baja, ya que pueden ejecutarse con una cierta periodicidad sin afectar la experiencia de usuario.
- Tareas de inicialización tienen la prioridad más baja, ya que solo corren al principio y luego se eliminan.

La organización de prioridades permite que el sistema sea:

- Responsivo
 - Eficiente
 - Predecible
-

IV.CONCLUSIONES

Con la realización y desarrollo de este trabajo, hemos adoptado conocimientos en la utilización de FreeRTOS y manejo de periféricos en lenguaje C. Además reforzamos la capacidad de trabajar en equipo, pudiendo dividir las tareas y desarrollar las consignas de manera más eficiente.

RECONOCIMIENTOS

En esta sección destacaremos la colaboración que nos brindaron los profesores Fabrizio Carlassara y Sergio Medina.

REFERENCIAS

- [1] Proyecto integrador de ejemplo utilizando FreeRtos.
- [2] Conocimientos desarrollados y actividades realizadas en el curso.